

GenAI Scenario-Based Interview Preparation Guide

50 Real-World Questions with Detailed Answers

Across eight critical domains: Foundations, Prompt Engineering, Embeddings,
RAG, Agents, Fine-Tuning, Deployment, and System Design.

Euron

euron.one | euronsystems.com

Prepared by Sudhanshu Kumar

Curated for engineers preparing for senior GenAI roles.

Section 1 — Foundations of Generative AI

This section tests whether you can choose, evaluate and reason about large language models as engineering components, not just as products. Strong answers come from making trade-offs explicit, not from listing features.

1. Your CEO wants to build an AI-powered customer support assistant. How would you decide whether to use GPT, Claude, Gemini, or an open-source model?

Framing this as a single-axis choice is the trap. The right answer is a decision matrix across five axes, scored against the business context.

Step 1 — Define the success metric. Customer support is measured by deflection rate, CSAT, average handle time, and cost per resolved ticket. Before you can pick a model, you need a target on each.

Step 2 — Establish the constraints. Where does the data live? Is the company in a regulated industry (BFSI, healthcare)? Is there a hard latency SLA? How much call volume per month?

Step 3 — Compare on the relevant axes:

- **GPT (OpenAI / Azure OpenAI):** Strong general capability, excellent function calling, mature SDK ecosystem. Azure OpenAI gives enterprise data controls and regional deployment. Best when you want a balanced, well-supported default.
- **Claude (Anthropic):** Strong reasoning, long context (200K+ tokens), excellent instruction following and safety behaviour. Often the best choice when tickets are nuanced, when policy adherence matters, or when you need to load long policy/product context.
- **Gemini (Google):** Native multimodal, strong on image and document understanding, deep integration with Google Workspace and Vertex AI. Useful when support tickets include screenshots or PDFs.
- **Open-source (Llama 3.x, Mistral, Qwen):** Lower marginal cost at high volume, full data control, on-prem deployment, ability to fine-tune. You pay for it in MLOps, GPU procurement and a serving stack (vLLM, TGI).

Step 4 — Run a structured POC. Pick 200–500 representative tickets (a mix of common, complex, and edge cases). Run the same prompts through 3–4 candidate models. Score on accuracy, policy adherence, tone, latency, and cost. Bring the numbers to the CEO.

Practical recommendation. Start with a managed API (Claude or GPT) to ship fast, instrument everything, and once volume justifies it (typically >100M tokens/month), evaluate moving the easy 70% of traffic to a fine-tuned open-source model behind a router, while keeping the hard 30% on the frontier API.

2. A GenAI application is producing inconsistent answers for the same question. How would you investigate the issue?

Inconsistency in LLM systems almost always traces to one of five root causes. The interview answer should be a systematic elimination, not a guess.

1. Sampling parameters. Check temperature, top_p, top_k, and seed. If temperature is anything above 0 and seed is not pinned, variation is expected. Set temperature=0, fix the seed (where supported), and re-test.

2. Model version drift. Pinned model aliases like "gpt-4" or "claude-3-opus" silently re-point to newer snapshots. Always pin to a dated version like gpt-4-2024-08-06 or claude-opus-4-7. Check whether your last release pinned the version.

3. Prompt mutation. Inspect what the model actually received. Often, what looks like the "same question" gets concatenated with different system prompts, different RAG context, or different conversation history. Log the full final prompt for both runs and diff them.

4. Retrieval non-determinism. If RAG is involved, vector search can return results in different order on tied scores. Reranker outputs can also shuffle. Check whether retrieved chunks differ between runs.

5. Routing / fallback. Many production systems route across multiple models or fall back on rate limits. The user sees "the same question" but the request hit a different model. Inspect the routing logs.

Action plan. Add a request fingerprint (hash of system prompt + user prompt + context + model + parameters). Log it with every response. Differences will jump out. For high-stakes flows, also add output validation against a JSON schema so format inconsistency is caught before it reaches the user.

3. Your company wants to deploy an AI application in a highly regulated industry. What factors would you consider before selecting a model?

Regulated deployment is not a model decision, it is a procurement and compliance decision. The model is only one node in a much larger control surface.

Data and privacy

- **Data residency:** where is the inference compute, where are the logs stored, and does the model provider have a region within the regulator's jurisdiction (e.g. India for DPDP Act, EU for GDPR).
- **Training opt-out:** explicit contractual guarantee that prompts and completions will not be used to train future models.
- **Zero data retention:** many providers offer ZDR contracts where prompts are not logged at all.
- **PII handling:** redaction pipelines on input, output filters for accidental disclosure, and tokenisation for irreversible IDs.

Compliance and certification

- **Certifications required:** SOC 2 Type II, ISO 27001, HIPAA BAA (US healthcare), PCI-DSS (payments), FedRAMP (US gov), DPDP Act compliance (India).
- **Audit logs:** immutable logs of who called the model, what was sent, what was returned, retained for the regulatory period.
- **Right to explanation:** for credit, hiring or healthcare decisions, the EU AI Act and similar laws may require explainability of automated decisions.

Model behaviour

- **Bias and fairness evals:** documented evaluations on protected attributes relevant to the use case.

- **Hallucination control:** RAG grounding, citations, and refusal behaviour when uncertain.
- **Versioning and immutability:** ability to pin the exact model snapshot used for a given customer interaction, and to reproduce it months later for audit.

Commercial

- **Indemnification:** vendor indemnifies you against IP claims arising from model outputs.
- **SLA:** uptime and latency commitments with financial penalties.
- **Exit strategy:** data export and the cost of migrating to another model.

Practical pattern. For BFSI in India, Azure OpenAI in the India region with a custom VNet, or self-hosted Llama 3 on AWS Bedrock with a VPC endpoint, are the two most common compliant patterns today.

4. You have a budget of only \$500/month for inference. How would you choose the right model architecture and deployment strategy?

This is a unit-economics question. The answer is to back-calculate from the budget.

Step 1 — Estimate volume. Suppose the product needs to serve 50,000 conversations per month, average 4 turns each, average 800 tokens of input and 300 tokens of output per turn. That is 200,000 model calls and roughly 220M tokens total per month.

Step 2 — Pick the right tier. At 220M tokens, frontier models (GPT-4o, Claude Opus) are out — they would cost 5–10x the budget. Small-but-capable models like Claude Haiku 4.5, GPT-4o-mini, Gemini Flash, or Llama 3.1 8B are the realistic candidates. Their input/output token pricing typically lands the whole month in the \$80–\$300 range, leaving budget for evals and overflow.

Step 3 — Optimise the prompt-side cost. Most cost in production LLM apps is input tokens, not output. RAG instead of stuffing the full knowledge base, prompt caching where the provider supports it, semantic caching of identical or near-identical queries (a Redis layer with embedding similarity), and structured outputs to avoid verbose generations.

Step 4 — Cascade. Route easy queries (classification, intent detection, simple Q&A) to the cheapest model. Escalate only the hard tail to a larger model. A well-tuned cascade often cuts spend 40–70% with no quality loss.

Step 5 — Consider self-hosting only above a threshold. Below ~50M tokens/month, a managed API is cheaper than running your own GPU. Above ~500M tokens/month, an open-source model on vLLM on a single A100 starts to pay back. At \$500/month, stick with the API.

Recommendation. Claude Haiku or Gemini Flash with semantic caching, prompt compression, RAG, and a cascade. Budget alerts at \$300 and a hard cap at \$475. This pattern reliably serves 50K–200K conversations per month within \$500.

5. A user asks why a 70B model performs better than a 7B model. How would you explain it using a practical business example?

The interviewer is testing whether you can translate machine learning to a non-technical stakeholder. Lead with an analogy, then ground it in a business example.

Analogy. Think of parameters as the number of distinct patterns and associations a model has internalised during training. A 7B parameter model is like a bright junior associate who has read the standard textbooks. A 70B model is like a senior consultant who has seen ten times more cases and can connect ideas across them.

Business example. Imagine a B2B SaaS support assistant answering: "My customer is on the Enterprise plan in Germany, signed in 2022 under our old MSA, and they want to add SSO mid-contract. What is the procedure and what are the commercial implications?"

- **The 7B model** can identify each phrase (SSO, MSA, Enterprise plan) and answer them individually but tends to miss how the 2022 MSA, the German GDPR context, and mid-contract amendments interact. Its answer is usually plausible but locally correct and globally wrong.
- **The 70B model** holds more of the cross-references in mind at once. It is more likely to flag that the 2022 MSA had a different data processing addendum, that SSO triggers a security review, and that mid-contract changes typically need a co-terminus amendment.

Why this happens. More parameters give the model a larger representational space, so subtle dependencies (terminology, jurisdiction, time-period) can be encoded without collapsing into a generic answer. Larger models also tend to follow long, multi-constraint instructions better.

Trade-off. 70B inference costs roughly 10x and is 3–5x slower. So the right architecture in practice is to use the 70B model only on the hard tail of queries, with the 7B handling the well-defined majority — a cascading pattern that captures most of the quality gain at a fraction of the cost.

Section 2 — Prompt Engineering

This section assesses whether you can engineer reliability into systems where the underlying component (the LLM) is stochastic. Senior interviewers want to see prompts treated as software: versioned, tested, and instrumented.

6. A chatbot frequently ignores business policies and generates unwanted responses. How would you redesign the prompts?

Treat this as a defect investigation, not a rewrite reflex. The same prompt may be ignored for several distinct reasons.

Step 1 — Reproduce and classify the failures. Collect 30–50 examples of the policy violation. Group them: is the model recommending competitors, quoting wrong prices, breaking tone, leaking information, going off-topic? Different failures need different fixes.

Step 2 — Audit the system prompt structure. Common problems: policies buried in the middle of a long prompt (LLMs attend less to the middle — the "lost in the middle" effect), policies stated as suggestions ("please try not to") instead of imperatives, contradictory rules, or policies that only kick in for unusual queries.

Step 3 — Restructure with a known-effective pattern:

- Open with role and identity ("You are the official support assistant for ABC Bank.").
- Follow with non-negotiable rules as a numbered list ("You must NEVER quote interest rates from memory. You must ALWAYS cite a knowledge base ID for any product claim.").
- Then add behavioural guidance (tone, refusal style, escalation triggers).
- End with the user message clearly delimited (XML tags or markdown fences).

Step 4 — Add few-shot examples of the model correctly handling each violation category. Two or three sharp examples often outperform a hundred lines of abstract rules.

Step 5 — Add a self-check step for high-stakes outputs: the model first drafts, then re-reads against the policy list, then either approves or revises. This costs an extra call but cuts violations dramatically.

Step 6 — Add a deterministic guardrail layer after the model. A regex/keyword filter and a small classifier (Llama Guard, an in-house BERT) catch the violations that slip through. Defence in depth beats any single perfect prompt.

7. You need an AI assistant to behave like a senior financial advisor. How would you structure the system prompt?

The structure matters more than the wording. A good financial advisor system prompt has six distinct blocks, in this order:

1. Identity and credibility anchor

"You are a CFA-charterholder financial advisor with 20 years of experience in Indian capital markets, specialising in retail wealth management. Your tone is calm, evidence-based, and conservative."

2. Scope and refusal policy

"You can discuss asset allocation, tax-efficient investing under Indian tax law, retirement planning, and broad market education. You must not recommend specific stocks, predict market direction, or give legal or accounting advice. For any of these, you politely redirect the user to a SEBI-registered advisor or a chartered accountant."

3. Mandatory inputs you must elicit

"Before giving substantive advice you must establish: age, current income range, dependents, time horizon, existing investments, and risk appetite. If any of these are missing, ask one clarifying question at a time."

4. Response format

Lock to a four-part structure: Situation Summary → Analysis → Recommendation → Risks & Caveats. This is the format real advisors use and it forces the model to acknowledge uncertainty.

5. Citation and grounding rule

"Any specific number (tax slab, return assumption, fund category) must come from the provided context. If it is not in the context, state that you need to verify and do not guess."

6. Mandatory disclaimer

Every substantive response ends with the regulator-required disclaimer ("This is educational information, not personalised investment advice...").

Why this works. Financial advice failures are usually not about model capability — they are about scope creep, premature recommendations, and missing disclaimers. The six-block structure makes each of those a hard gate.

8. Users are getting different answers from the same prompt. What prompt-engineering techniques would you apply?

Inconsistency at the prompt level (not the routing level — see Question 2) is fixed with four techniques applied in combination:

- **Determinism settings:** set temperature=0, top_p=1, and pin a seed where the provider supports it.
- **Structured output:** switch from free-text to a JSON schema (OpenAI structured outputs, Claude tool use, Gemini response_schema). The schema constrains both content and format.
- **Few-shot anchoring:** include 2–4 input/output examples in the prompt that demonstrate the exact format and reasoning style you want. The model learns to imitate them, reducing variance.
- **Output validation with retry:** after generation, validate against the schema or expected fields; on failure, re-prompt with the error. This converts soft variance into hard correctness.

A subtler source: if the prompt template is rendered from variables, even a trailing newline or a swapped Unicode quote can change the tokenisation and therefore the output. Snapshot the final rendered prompt as a string and hash it. Two requests with the same hash should produce the same output when temperature is zero.

When variance is desirable, for example in creative or recommendation generation, control it explicitly via temperature rather than letting it leak in. Document the intended variance.

9. Your AI assistant gives correct answers but the format changes every time. How would you enforce consistency?

Format drift is one of the most common — and most fixable — production issues.

Strongest fix: structured outputs. Define the output as a JSON schema and use the provider's structured output feature. OpenAI's `response_format` with `json_schema`, Anthropic's tool use, and Gemini's `responseSchema` all guarantee the model returns valid JSON in the shape you specified. This is more robust than asking nicely in a prompt.

Second-strongest fix: few-shot examples. Include 2–3 examples in the prompt showing the exact format. The model will mimic them. This is the right tool when you cannot use structured outputs (e.g. you need markdown or a specific natural-language format).

Third: prefix injection. Start the assistant's response for it. For example, end the prompt with: "Respond using exactly this format:\n```\njson\n{\n \"summary\": \" — the model will continue from there.

Fourth: post-process with templates. Have the model produce only the variable parts, then assemble the final output with a deterministic template (Jinja, string formatting). The model never controls the layout.

Operational layer: validate every output against the schema and reject/retry on failure. Log format-failure rate as a metric. If it rises, alert.

10. A legal assistant must always cite sources while answering. How would you design the prompt?

The system has to make citation impossible to skip. That requires changes on three layers: retrieval, prompt, and validation.

Retrieval layer. Build a RAG pipeline that returns each chunk with stable identifiers: source name, document ID, section, page, and the actual passage text. Citations are only as good as the IDs you give the model.

Prompt structure.

- System prompt: "You are a legal research assistant. Every factual statement in your answer must be supported by a citation in the format [DOC_ID §section, p.X]. If a statement is not supported by the provided context, you must say 'I could not find authority for this point in the provided materials.' Do not cite from general knowledge."
- Provide the retrieved chunks in a clearly delimited <context> block, each chunk prefixed with its ID.
- Force structured output: an array of (claim, citation_ids, supporting_quote) tuples, then a synthesis paragraph.

Validation layer. After generation, verify that every claim object has at least one citation_id and that each ID exists in the retrieved set. If not, reject the response and retry with a stronger instruction. This is non-negotiable for legal use cases.

UX layer. Surface citations as clickable links that open the exact passage. This both helps the user and creates a feedback loop where bad citations get reported.

11. An AI recruiter needs to generate interview feedback in a specific company template. How would you achieve this?

Template fidelity is best solved by separating content generation from layout.

Step 1 — Define the template as a schema. Identify each variable section: candidate name, role, interviewer, date, competency scores (e.g. communication, technical depth, problem solving, culture fit, each 1–5), strengths (bullet list), gaps (bullet list), overall recommendation (Strong Hire / Hire / No Hire / Strong No Hire), and rationale.

Step 2 — Use structured outputs. Define a JSON schema with these exact fields and pass it to the LLM. The model fills the fields based on the interview transcript and notes. This guarantees every field is present.

Step 3 — Render through a fixed template. Use a deterministic renderer (Jinja2, docx-template, an HTML template) to insert the JSON into the company's branded format. The model never produces the template — it only produces the content.

Step 4 — Calibrate with few-shot examples. Include 2–3 anonymised real feedback reports in the prompt so tone, granularity and rationale length match company norms.

Step 5 — Add quality gates. Validate that scores are within range, that the recommendation matches the score distribution (no "Strong Hire" with all 2s), and that rationale length is within bounds. Auto-reject and regenerate on failure.

Why this beats prompting alone. Prompting the model to follow a template format works 80% of the time. The remaining 20% are exactly the cases where a recruiter notices and loses trust. Schema + template gets you to 100%.

12. Your prompt works perfectly in testing but fails in production. What could be the reasons?

This gap is one of the most common pain points in shipping LLM systems. The likely causes form a checklist:

- **Distribution shift.** Test data was clean and representative of the happy path. Production has typos, multilingual input, copy-pasted formatting, emoji, URLs, very short or very long inputs. Build the eval set from production samples.
- **Prompt injection.** Users include text that the model interprets as instructions. Defence: separate instruction and data channels using XML tags or markers, run an input classifier, and never let user input override system directives.
- **Context window overflow.** In production, conversation history or RAG context can push the prompt past the window. Critical instructions at the start of the system prompt get silently truncated. Fix: enforce a max context budget and place critical rules at both the start and end.
- **Model variant or version.** Production may route through a different model (A/B, fallback, regional). Pin the version explicitly in production and assert it in the logs.
- **Tool / RAG schema drift.** A backend team changed the tool response shape, or the RAG index moved. The prompt still works in isolation but the inputs to it changed. Contract-test the integrations.
- **Concurrency / rate-limit fallback.** Under load, the gateway falls back to a smaller model or returns partial responses. Log the actual model that served each call.
- **Locale and encoding.** Production receives UTF-8 with combining characters, BOM markers, right-to-left text. Test these.

Operational fix: shadow-mode logging (run new prompts against production traffic without serving the result) for at least a week before promotion, plus an evaluation set built from production samples updated monthly.

Section 3 — Embeddings and Vector Databases

This section tests retrieval engineering. The mistakes here are usually subtle — bad chunking, the wrong distance metric, an embedding model that does not match the domain — and they degrade RAG quietly. The strongest answers demonstrate diagnostic thinking.

13. Your company has 100,000 PDF documents. How would you build a semantic search system for employees?

100K PDFs is a midsize corpus — large enough that naive approaches fail, small enough that a single well-built pipeline can serve it on modest infrastructure.

Ingestion pipeline

- **Extraction:** use a strong PDF parser (PyMuPDF, Unstructured, or Azure Document Intelligence for scanned PDFs with OCR). Preserve structural cues — headings, tables, page numbers.
- **Cleaning:** strip headers, footers, page numbers from body text, but keep them in metadata.
- **Chunking:** semantic chunking with overlap. Default to 500–800 tokens per chunk with 15% overlap, but split on heading boundaries when present. Never split mid-table or mid-list item.
- **Metadata enrichment:** for each chunk, capture source filename, page range, section heading, department, last modified date, document classification (public / internal / confidential), and the ACL — who is allowed to see this.

Embedding

- **Model choice:** text-embedding-3-large, BGE-large, or Cohere Embed v3 are strong defaults. If documents are domain-specific (legal, medical), evaluate a fine-tuned domain embedding.
- **Batching:** embed in batches of 100–500 for cost efficiency. Cache embeddings keyed by chunk hash so re-runs are cheap.

Vector store

- **At 100K docs × ~10 chunks each = ~1M vectors.** This fits comfortably in pgvector with HNSW, Qdrant, Weaviate, or a small Pinecone pod. Start with pgvector if you already use Postgres.
- **Index:** HNSW with M=16, ef_construction=200, ef_search tuned per query.
- **Hybrid search:** combine BM25 (Postgres GIN or OpenSearch) with vector search and fuse with Reciprocal Rank Fusion. Pure vector misses exact keyword matches; pure keyword misses semantic intent.

Retrieval and ranking

- **Stage 1 — recall:** top-50 from hybrid search.
- **Stage 2 — rerank:** Cohere Rerank or BGE-reranker to bring the top-5 to the top.
- **Stage 3 — filter:** apply ACL at query time — never trust pre-filtering at index time alone.

Surface

Search UI shows snippet, source filename with page link, and last-modified date. Add a "feedback" thumb for each result so retrieval quality can be tracked.

14. Users complain that relevant documents are not appearing in search results. How would you debug the embedding pipeline?

Debug retrieval failures by collecting ground truth, then walking through each stage of the pipeline to find where the relevant document drops out.

Step 1 — Build a tiny ground truth set. Take 20–30 failing queries and ask the user to point to the document they expected. This is the only way to debug retrieval objectively.

Step 2 — Bypass the pipeline and test directly. For each query, manually retrieve the expected document. Confirm it exists in the index. If it does not, the problem is ingestion.

Step 3 — Walk the pipeline:

- **Was the document chunked correctly?** Inspect the chunks. The relevant sentence may have been split across two chunks, or buried in a chunk dominated by unrelated text.
- **Was the chunk embedded?** Confirm the embedding is in the vector store and has the right dimensionality.
- **Does the query embedding match the chunk embedding?** Compute cosine similarity manually. If similarity is low, the embedding model may not be capturing the user's intent — try query rewriting, HyDE, or a stronger embedding model.
- **Is the chunk in the top-K?** If similarity is high but the chunk is not in top-K, increase K or check whether a filter is excluding it.
- **Is the chunk being reranked away?** Inspect reranker scores. Sometimes a reranker promotes plausible-but-wrong chunks.

Common root causes. Chunking too aggressive (relevant text fragmented), embedding model mismatch (e.g. using an English-only model on Hindi text), missing metadata filter (the document was tagged for a department the user does not have access to), or query vocabulary mismatch (user uses jargon that the embedding model has not seen).

Fix patterns. Add query expansion (LLM rewrites query into 3 variants), enable hybrid search to catch lexical matches, or fine-tune the embedding model on in-domain query-document pairs.

15. You are building a legal document search engine. What chunking strategy would you use and why?

Legal text is the worst case for naive chunking. Meaning is encoded across sections, references resolve to other clauses, and a single split in the wrong place inverts the intent.

Principle. Chunk by legal structure, not by token count. Legal documents have a natural hierarchy — Act, Chapter, Section, Sub-section, Clause — and that hierarchy carries meaning.

Recommended approach:

- **Primary unit = section.** Parse the document into its structural sections. A section is typically self-contained enough to answer a question about its subject.
- **Secondary unit = sub-section.** If a section is over 1,500 tokens, split at sub-section boundaries but include the parent section heading in each child chunk so the context is preserved.

- **Tertiary fallback = sliding window.** For very long sub-sections (e.g. detailed schedules), use a 1,000-token window with 25–30% overlap so cross-references survive splitting.

Metadata you must capture per chunk:

- Document title (e.g. "Companies Act, 2013")
- Section path ("Chapter IV → Section 42 → Sub-section (4)")
- Effective date and amendment history
- Jurisdiction
- Cross-references mentioned in the chunk (parsed into a separate list)

Why not arbitrary 500-token chunks. A 500-token chunk that splits the middle of a definition clause makes the clause unsearchable. Worse, an LLM reading that chunk will confidently produce a plausible but wrong answer because the qualifier was lost. Legal interpretation is structural; chunking must respect that structure.

Beyond chunking. For cross-references, build a graph alongside the vector store. When a chunk is retrieved, optionally pull the chunks it references, with depth=1, so the LLM sees the connected clauses too.

16. The vector database response time suddenly increases from 100ms to 2 seconds. How would you investigate?

Treat this as a production incident. Restore service first if it is user-facing, then investigate.

Step 1 — Localise. Is the latency in the vector DB itself, or upstream (embedding the query) or downstream (reranker, LLM)? Add timing logs at each stage and look at the breakdown.

Step 2 — Check the obvious.

- Recent deploys: a new index config, a new embedding dimension, a changed similarity metric, a different K.
- Recent data growth: vector count, average vector length, index size on disk.
- Recent query pattern change: are filters now more selective or less? Is K higher?
- Infrastructure: CPU, memory, disk I/O, network. A noisy neighbour on a shared instance can explain sudden 20x latency.

Step 3 — Inspect the index.

- HNSW degrades when many vectors have been deleted but not vacuumed. Rebuild the index.
- ef_search may be too high for the current index size.
- If using IVF, the number of probes may need re-tuning after data growth.
- Disk-resident indexes (e.g. DiskANN) become slow when working set exceeds RAM cache.

Step 4 — Check metadata filtering. Many vector DBs apply filters post-search, so a highly selective filter forces them to retrieve many candidates before finding K that match. Pre-filtering indexes (Qdrant payload index, Weaviate filter index) restore performance.

Step 5 — Check the surrounding system. Is the connection pool exhausted? Is there a thundering herd from a downstream service? Is the embedding API itself slow (the query embedding is part of total latency)?

Mitigations while investigating. Add a Redis cache for hot queries, scale the DB instance, and shed load on non-critical clients.

17. A customer asks why vector search is better than keyword search. How would you explain it with an example?

Frame this as a strengths-and-trade-offs comparison, then anchor with an example the customer will recognise.

Keyword search matches on the exact words you type. It is fast, predictable, and good for retrieving by name or ID. Its weakness is that it cannot recognise meaning. A search for "cancel my plan" will not match a document titled "How to terminate your subscription."

Vector (semantic) search represents queries and documents as numerical vectors in a space where similar meanings are close together. So "cancel my plan" and "terminate subscription" land near each other and are matched.

Concrete example. A user types into a banking support search: "my card got eaten by the ATM." Keyword search needs the document to literally contain "eaten" — unlikely. Vector search retrieves "card retention at ATM — what to do," which is the correct article, because the embedding model has learned that those phrases describe the same situation.

Real recommendation. Use both. Hybrid search (BM25 keyword + vector) consistently beats either alone in production. Keyword handles exact identifiers (account numbers, product codes) where embeddings struggle; vectors handle paraphrases and intent. A reranker on top of both gives the best result.

18. Your retrieval quality drops after switching embedding models. What steps would you take?

This is a known migration pitfall. Embeddings from one model are not comparable to embeddings from another.

Step 1 — Confirm the corpus has been fully re-embedded. The single most common cause is that only new documents were embedded with the new model, while old documents still hold the old embeddings. Cosine similarity between two different embedding spaces is meaningless. The whole corpus must be re-embedded.

Step 2 — Re-tune retrieval parameters. Different embedding models have different similarity distributions. A cosine threshold of 0.75 that worked for OpenAI ada-002 may correspond to 0.45 for BGE-large. Recalibrate K, the similarity threshold, and any reranker scores.

Step 3 — Verify the model fits the domain. The new model may be technically better on benchmarks but worse for your text — common with code, legal, medical, or multilingual content. Run a side-by-side eval on your own ground truth, not on MTEB.

Step 4 — Check distance metric and normalisation. Some models output normalised vectors and assume cosine; others assume dot product. A mismatch silently destroys retrieval.

Step 5 — Check truncation. Models have different max input lengths. If your chunks were 8K tokens and the new model truncates at 512, you are only embedding the start of each chunk.

Step 6 — Decide whether to roll back. If the new model loses on your eval set after proper migration, roll back and document why. Better embeddings on a benchmark do not imply better retrieval on your data.

Section 4 — Retrieval Augmented Generation (RAG)

RAG is where most enterprise GenAI projects live and die. Strong answers in this section show that you treat RAG as a pipeline of independently measurable stages, not a single black box.

19. A customer support bot hallucinates product information even though the knowledge base contains the correct answer. How would you solve this?

Hallucination despite correct KB is almost always a generation-side issue, not a retrieval-side one. Diagnose in two stages.

Stage 1 — Confirm retrieval is actually returning the right chunk. Log the retrieved chunks for the failing queries. If the right chunk is not in the top-K, the problem is retrieval (see Question 14) and no amount of prompt engineering will fix it.

Stage 2 — If the right chunk is being retrieved but the answer is still wrong, fix generation:

- **Tighten the grounding instruction.** "Answer using ONLY the information in <context>. If the answer is not present in <context>, say 'I could not find this in our knowledge base.' Do not use your prior knowledge."
- **Force citations.** Require the model to quote the supporting text and the chunk ID. Models hallucinate less when they have to point at evidence.
- **Lower temperature.** Set to 0 for factual answers.
- **Use a stronger model on the generation step** even if retrieval uses a smaller one. Hallucination correlates with model size.
- **Address "lost in the middle".** If many chunks are passed, the relevant one in the middle gets ignored. Limit to 3–5 high-quality chunks and place the most relevant at the start and end.
- **Add a self-check.** After generation, ask the model: "Is every claim in this answer supported by the provided context? If not, list the unsupported claims." Retry if any are unsupported.
- **Add a verifier model.** A separate, smaller model classifies (claim, context) → (supported / unsupported). Reject responses with unsupported claims.

Operational pattern. Maintain a regression set of historically-hallucinated queries and run it on every prompt change. Hallucination rate should be a tracked metric in production.

20. Your RAG system performs well on small documents but poorly on large enterprise manuals. How would you improve retrieval?

Large documents fail for predictable reasons. The fix is structural, not parameter tuning.

Problem 1 — Chunking destroys hierarchy. An enterprise manual has chapters, sections, sub-sections. Flat chunking loses this. Solution: hierarchical chunking that captures the path (Chapter → Section → Sub-section) as metadata so retrieval can use it.

Problem 2 — One chunk is not enough context. Manuals build up an explanation across several paragraphs. Solution: parent-child chunking. Index small chunks for precise retrieval, but when a child chunk is retrieved, return its parent (the full section) to the LLM. This is sometimes called "small-to-big" retrieval.

Problem 3 — The user's query does not match document vocabulary. Manuals use formal language; users use shorthand. Solution: multi-representation indexing. For each chunk, also index (a) an LLM-generated summary, (b) a list of hypothetical questions the chunk answers (HyDE-like), and (c) keywords. Match the query against all three.

Problem 4 — Cross-section dependencies. Answer needs information from Section 4 and Section 12. Solution: query decomposition — the LLM rewrites the user question into 2–4 sub-questions, retrieves for each, then synthesises.

Problem 5 — The manual covers multiple products / regions / versions. Solution: a routing layer. Before retrieval, classify which product or version applies, then filter to that subset. This dramatically improves precision and speed.

Problem 6 — The manual updates frequently. Solution: track version, deprecation, and effective dates as metadata so retrieval never returns superseded sections.

21. A user asks a question requiring information from multiple documents. How would your RAG architecture handle this?

Single-shot retrieve-then-generate fails when the answer requires synthesis across documents. The architecture must handle this explicitly.

Pattern 1 — Query decomposition. Use a small LLM call to split the user question into independent sub-questions. Retrieve for each. Pass all retrieved context to the final generation. Example: "How does our refund policy compare to our shipping policy for international orders?" decomposes into "What is our refund policy for international orders?" and "What is our shipping policy for international orders?"

Pattern 2 — Iterative / agentic RAG. The LLM is given retrieval as a tool. It plans, retrieves, reads, decides whether it has enough information, and either answers or retrieves more. Frameworks: LangGraph, LlamaIndex agents. More flexible, higher latency.

Pattern 3 — Self-RAG / reflective retrieval. The LLM emits special tokens ("retrieve", "sufficient", "insufficient") that drive a controller. It retrieves only when needed and stops when confident.

Pattern 4 — Graph RAG. When documents reference each other (e.g. policies that cross-reference other policies, legal documents with citations), build a knowledge graph. Start with vector retrieval, then traverse the graph one or two hops to gather related context.

Operational considerations.

- Cap iterations to control cost and latency.
- Track the final set of sources used and surface all of them to the user.
- Cache common decompositions.
- Fall back to single-shot retrieval for simple queries — do not pay the multi-hop cost for everything.

22. The retrieval stage returns relevant documents, but the LLM still generates incorrect answers. How would you debug the system?

This is a generation-side bug, not a retrieval bug. Walk through five common causes.

1. The model is ignoring the context. Inspect the prompt — is the context clearly delimited, and does the instruction explicitly say to use only the context? Reorder the prompt so the context is close to the question, and the instruction to use it is repeated near the end.

2. "Lost in the middle" effect. When 10+ chunks are passed, LLMs attend more to the start and end. Reduce K, rerank, and place the highest-scoring chunk near the end of the context block.

3. Context is relevant but not sufficient. The retrieved chunks contain related information but not the specific answer. The LLM extrapolates. Tighten the instruction to say "insufficient information" rather than guess, and improve retrieval to return more precise chunks.

4. The context contradicts itself. Two retrieved chunks say different things (older policy and newer policy, two regions). The LLM picks one arbitrarily. Solution: deduplicate by recency or jurisdiction at retrieval time, or pass metadata so the model can reason about which applies.

5. The model has strong prior beliefs that override the context. Especially on common topics. Solution: use a stronger model, lower temperature, and explicitly tell the model the provided context may differ from its prior knowledge and to defer to the context.

Diagnostic technique. Build a small panel of failing queries with the exact retrieved context. Run them through multiple models (small and large) and multiple prompt variants. The pattern of failure across this grid usually identifies the root cause.

23. Your company wants to build a chatbot over SharePoint, PDFs, and Confluence pages. How would you design the RAG pipeline?

Three sources, three connectors, one unified index. The architecture has to handle source-specific quirks while presenting a single search surface.

Ingestion layer

- **SharePoint:** Microsoft Graph API with delegated permissions. Pull document libraries, respect site permissions, and capture last-modified for incremental sync.
- **PDFs:** Unstructured.io or Azure Document Intelligence for high-fidelity extraction including tables and OCR for scanned files.
- **Confluence:** Atlassian REST API. Pull pages including labels, space, and version history.
- **Scheduling:** Initial backfill plus incremental sync every 15–60 minutes via webhooks where available, polling otherwise. Treat ingestion as an ETL pipeline (Airflow, Prefect, or a simple cron + queue).

Normalisation layer

- **Convert each source to a common document model:** title, body (markdown), structured metadata (source, ID, URL, author, last-modified, ACL).
- **Preserve permissions:** carry SharePoint ACLs, Confluence space restrictions, and user-group mappings into the metadata. Permissions are enforced at query time, not at index time.

Chunking and embedding

- Source-aware chunking — PDFs by section, Confluence by heading, SharePoint mixed.
- One embedding model across all sources for comparable similarity.
- Multi-vector: chunk + summary + hypothetical questions.

Storage and retrieval

- A single vector store (Qdrant, Weaviate, pgvector) with source as a metadata filter.
- Hybrid search (BM25 + vector) + reranker.
- ACL filter applied at query time using the requesting user's group memberships.

Generation and UI

- RAG prompt with citations linking back to the source URL — clicking opens the SharePoint file, Confluence page or PDF at the right page.
- Source diversity rule: do not let the answer cite only one source if multiple are relevant.

Operational concerns

- Re-index when source schemas change.
- Soft-delete: when a Confluence page is deleted or a SharePoint file is moved, remove its chunks.
- Sensitive content flagging: pages tagged "confidential" should be visible only to users with the right group.

24. Search quality improves after increasing chunk size but latency becomes worse. How would you balance the trade-off?

This is a classic precision-recall-cost trilemma. Resolve it with architecture, not by picking a single chunk size.

Pattern 1 — Small-to-big (parent-child) retrieval. Index small chunks (200–400 tokens) for precise retrieval, but when a small chunk is retrieved, fetch its parent (1500–2500 tokens) and pass that to the LLM. You get retrieval precision and generation context in one move.

Pattern 2 — Two-stage retrieval. First stage retrieves top-50 small chunks. Second stage reranks them. Only the top 3–5 are expanded to their parents and sent to the LLM. Latency is dominated by the cheap small-chunk search.

Pattern 3 — Adaptive chunk size by document type. FAQs and short policies → small chunks. Long manuals and contracts → larger chunks. Auto-detect by section length.

Pattern 4 — Caching. If many users ask similar questions, cache the assembled context block (or even the final answer) keyed on a normalised query embedding. Latency drops to near-zero on cache hits.

Pattern 5 — Streaming and progressive disclosure. Begin streaming the LLM's answer as soon as the first chunks are retrieved; refine in the background if needed. The perceived latency improves even when the total latency does not.

Measurement. Track end-to-end latency at p50 and p95, retrieval recall@k on your eval set, and the LLM input token count. Decide on the cheapest configuration that meets your quality bar — not the most accurate one.

25. Users are asking questions containing synonyms and industry jargon. How would you improve retrieval quality?

This is a vocabulary-mismatch problem. The user and the documents do not share the same words for the same concept. Five techniques in combination usually solve it.

1. Hybrid search. Pure vector search handles synonyms but can miss exact jargon. Pure BM25 handles jargon but misses synonyms. Combining them with Reciprocal Rank Fusion is the most reliable first step.

2. Query expansion with an LLM. Before retrieval, rewrite the user query into 3–5 paraphrases that include both layman terms and likely jargon. Retrieve for each and union the results. Cost: one cheap LLM call. Benefit: significant recall improvement.

3. HyDE (Hypothetical Document Embeddings). Have the LLM draft a hypothetical answer to the query. Embed that draft and retrieve documents similar to it. The draft contains the jargon the user did not use.

4. Domain-specific embeddings. If the corpus is heavily jargon-based (legal, medical, semiconductor), use or fine-tune a domain-specific embedding model. General-purpose embeddings can be weak on specialised vocabulary.

5. Synonym dictionary in the indexing pipeline. Maintain a curated list of synonyms ("laptop = notebook = portable PC"). At index time, expand chunks; at query time, expand queries. This is unglamorous but extremely effective when the vocabulary is bounded.

6. Multi-representation indexing. For each chunk, also store a generated summary that uses common language, and a list of "what questions could this chunk answer" phrased in user vocabulary. Embed and index those too.

26. Your RAG system must support 10 million documents. What architectural changes would you make?

At 10M documents (often 100M+ chunks), the system must move from a single-node design to a distributed one. The bottlenecks shift from accuracy to throughput, cost, and operational complexity.

Vector store

- **Sharded vector DB:** Pinecone with multiple pods, Qdrant cluster, Weaviate multi-node, or Milvus. Choose ANN parameters (HNSW M, ef) that fit memory per shard.
- **Memory vs disk:** 100M vectors at 1024 dims × 4 bytes = ~400 GB. Either hold in RAM across nodes (expensive, fast) or use disk-resident indexes like DiskANN with smart caching.
- **Quantisation:** use scalar or product quantisation (PQ) to cut memory 4–8x with modest accuracy loss. Pair with a re-ranking step on the full-precision vectors for the top-K.

Retrieval architecture

- **Two-tier retrieval.** Tier 1: route the query to a small set of relevant shards / partitions using metadata (department, product, language) or a learned router. Tier 2: search those shards only. This is the single biggest win at scale.
- **Pre-filtering indexes.** Build payload indexes on metadata so filtered ANN queries do not have to retrieve thousands of candidates.
- **Multi-stage rerank:** ANN top-200 → cheap cross-encoder top-30 → expensive cross-encoder top-5.

Ingestion pipeline

- **Streaming:** Kafka or equivalent to decouple producers from indexers.

- **Workers:** horizontally scaled embedding workers (often GPU). Batch sizes tuned for the embedding model.
- **Idempotency:** every document has a stable ID; re-ingesting must not duplicate.
- **Backfill vs incremental:** separate priority queues.

Cost and operations

- **Cache hot queries** with semantic caching (vectorise the query, look up similar past queries with high similarity, return their answers).
- **Tier storage:** hot vectors in RAM, warm on SSD, cold archived. Most queries hit the hot tier.
- **Monitoring:** p95 retrieval latency per shard, recall@k on a ground-truth set sampled regularly, cost per query.
- **Replicas:** read replicas for query traffic, write to primary for ingestion.

Trade-offs to call out in the interview. At this scale, accuracy and latency are in tension. Approximate search and quantisation give you the throughput at the cost of recall. The architecture must explicitly choose where to spend on accuracy (the reranker on the top-K) and where to accept approximation (the ANN candidate set).

Section 5 — Agents and Tool Calling

Agentic systems compound the failure modes of LLMs with the failure modes of distributed systems. Strong answers here show that you respect both — orchestration, state, observability, and security all matter as much as prompting.

27. A travel-planning agent needs weather data, flight data, and hotel data. How would you design the agent workflow?

Design this as a planner-executor with a clear state model. The mistake to avoid is wiring tools in a linear chain — real travel planning has dependencies and parallelism.

Tools

- `get_weather(destination, date_range)` → forecast
- `search_flights(origin, destination, dates, preferences)` → list of flights with prices
- `search_hotels(location, check_in, check_out, preferences)` → list of hotels
- `get_attractions(destination, interests)` → list of activities
- `create_itinerary(plan)` → final formatted output

Workflow

Step 1 — Intent extraction. From the user's natural request, the planner extracts: origin, destinations, dates, traveller count, budget, preferences (beach / city / culture), and constraints (visa, dietary).

Step 2 — Plan. The planner builds a directed graph of tool calls. Weather can be fetched in parallel with flight search. Hotel search depends on which city is chosen (which depends on flight availability and weather).

Step 3 — Execute. An executor runs the graph: parallelise independent calls, sequence dependent ones. Each tool result is added to the agent's working memory.

Step 4 — Reason and iterate. After the first round, the agent evaluates: are weather conditions acceptable, are flights within budget, is the hotel near the activities? If not, re-plan (try alternative dates, alternative cities).

Step 5 — Synthesise. Once the agent has a viable plan, it generates a structured itinerary with options and rationale.

Implementation notes

- **Framework:** LangGraph or a custom state machine. Avoid a single "call the model in a while loop" pattern; it is hard to debug at scale.
- **Budget control:** cap the number of tool calls per session and total token spend.
- **Confirmation gate:** before any booking tool is invoked, present the plan to the user for explicit approval.
- **Observability:** log every tool call with inputs, outputs, latency, and the reasoning trace.
- **Fallbacks:** if a tool fails, the agent should know how to back off (e.g. if no direct flights, try one-stop).

28. An AI assistant needs to book meetings, send emails, and update CRM records. How would you implement tool calling?

These are mutating operations on systems of record. The bar for correctness, idempotency, and auditability is much higher than for read-only search.

Tool definitions

- Each tool gets a strict JSON schema (description, parameters with types, required fields, enum constraints).
- Mutating tools include an `idempotency_key` parameter that the agent must generate (UUIDv4). The backend uses this to deduplicate.
- Return values include a verifiable confirmation (event ID, message ID, record ID) and a deep link.

Auth and identity

- Tools run on behalf of the user, with the user's OAuth token. Never use a service account with broad access.
- Scope per tool — `calendar.write` does not imply `mail.send`.
- Refresh tokens stored encrypted, rotated on schedule.

Execution flow

- 1. Intent and slot filling:** the agent confirms it has all required parameters (attendees, time, duration, location). Missing slot → ask one question at a time.
- 2. Dry-run preview:** the agent shows what it is about to do ("I am going to send an email to X with subject Y" or "I am going to create a meeting Friday 3 PM with A and B"). For destructive or external-facing actions, an explicit user confirmation is required before execution.
- 3. Execute via the tool.** Pass the idempotency key. Capture the confirmation.
- 4. Post-action verify.** Read back the created object to confirm it matches the intent (e.g. retrieve the calendar event, confirm attendees and time).
- 5. Log the action** to an immutable audit table: who, when, what tool, inputs, output, `idempotency_key`, success/failure.

Failure handling

- Transient errors (HTTP 5xx, rate limits): retry with exponential backoff.
- Permanent errors (4xx other than 429): surface to the user with the error message, do not silently swallow.
- Partial success (some attendees invalid): report explicitly which parts succeeded.

Safety patterns

- Never invoke a destructive tool (delete, cancel, send to external recipient) without explicit user confirmation.
- Maintain an allow-list of recipient domains for outbound email when the agent operates autonomously.
- CRM writes should write to a staging field first if the change is significant; let the user promote.

29. Your agent repeatedly calls the same tool in an infinite loop. How would you prevent it?

Tool loops are a structural problem in the agent design. Fix them in three layers.

Hard limits (the safety net). Cap maximum iterations per session (e.g. 15 tool calls) and maximum total token spend. When hit, the agent must terminate with a graceful message rather than continue. This is non-negotiable in production.

Cycle detection (the diagnostic). Hash each (tool_name, normalised_arguments) call. If the same hash appears twice in a session, intervene. The intervention can be a system message to the model: "You have already called this tool with these arguments. The previous result was X. Either use that result or choose a different action."

Better planning (the root cause). Loops usually indicate the agent cannot extract what it needs from a tool's output, so it tries again. Common fixes:

- Improve the tool's output format — make the relevant fields easy to find.
- Provide a stronger plan prompt — "if a search returns no results, do not search again with the same query; broaden the query or ask the user".
- Move from a free ReAct loop to a Plan-and-Execute pattern, where the plan is generated up-front and execution is more deterministic.
- Add explicit termination conditions in the planner: "You are done when you have answer + citations + the user has been informed."

Observability. Log every loop with the model's reasoning trace. Most loop bugs are immediately obvious from the trace — the model is asking the same question and getting the same answer.

Fallback. If the agent has burned 80% of its budget without progress, hand off to a human or return a partial answer with an explanation.

30. A customer asks for a stock recommendation. The agent has access to financial APIs. How would you ensure the response is reliable?

This is a high-stakes domain with regulatory exposure (SEBI in India, SEC in US). Reliability here is multi-dimensional: data accuracy, model behaviour, disclosure, and compliance.

Data layer

- Pull from at least two independent sources (e.g. exchange data, a reputable provider). Cross-check key numbers before quoting.
- Always check the timestamp. Data older than market-close-minus-N minutes is flagged stale.
- Capture corporate actions (splits, dividends) — comparing raw prices across these is misleading.

Model layer

- Never let the model fabricate numbers. Tool outputs are the source of truth; the model only narrates them. Force quoting with citation.
- Frame outputs as analysis, not recommendations. The agent describes factors, scenarios, and risks; it does not say "buy" or "sell".

- Use a stronger model with low temperature for any quantitative reasoning.

Compliance layer

- Add mandatory disclaimers based on jurisdiction (SEBI requires explicit education-vs-advice framing in India).
- Detect when the question crosses into personalised advice and redirect to a registered advisor.
- Maintain a log of every recommendation-shaped output for regulatory audit.

User layer

- Show the data sources and timestamps.
- Show the model version that generated the response.
- Allow the user to flag a response as wrong; route flags into a review queue.

Bottom line. The reliability of a stock-related agent comes 30% from the model, 30% from the data, and 40% from the surrounding controls — citation, disclaimers, audit. Get the controls right and the model becomes a safe, useful interface to the data.

31. Multiple agents are collaborating on a business report. How would you divide responsibilities among them?

Multi-agent systems work when responsibilities are partitioned by skill, with a clear coordinator and a shared workspace. They fail when agents step on each other or when communication overhead exceeds the work.

Roles

- **Researcher:** gathers raw information from external sources (web, internal docs, APIs). Owns the citations.
- **Analyst:** transforms raw data into structured findings — tables, comparisons, trends. Owns the numbers.
- **Writer:** composes prose sections following a style guide. Owns voice and structure.
- **Reviewer / Critic:** checks for factual consistency with sources, logical coherence, and gaps. Has the power to reject and send back for revision.
- **Coordinator / Editor-in-chief:** owns the outline, assigns tasks, integrates contributions, and signs off the final report.

Architecture

- **Shared workspace** (a document, a database, or a structured scratchpad) where all agents read and write. This avoids passing everything through messages.
- **Message bus** for coordination (task assignments, status, escalations).
- **Explicit contracts:** each agent declares its inputs and outputs. The Coordinator routes work according to the contract.

Process

1. Outline. Coordinator drafts the report outline, identifies sections, and assigns each to a Researcher-Analyst-Writer trio.

2. Research phase. Researchers populate the workspace with sourced facts.

3. **Analysis phase.** Analysts transform facts into structured findings.
4. **Drafting phase.** Writers turn findings into prose.
5. **Review phase.** Reviewers check each section against the sources and the outline.
6. **Integration.** Coordinator integrates, resolves conflicts, and produces the final version.

Common pitfalls

- Too many agents — communication cost dominates. Start with the minimum set.
- No deadline / no budget — agents go in circles. Add per-section iteration caps.
- Identical models in all roles — diversity of model and prompt helps catch each other's blind spots. The Reviewer especially benefits from being a different (often stronger) model.

Frameworks. CrewAI, AutoGen, LangGraph, or a custom orchestrator. Choose based on observability needs — at production scale, the ability to inspect the conversation graph matters more than the framework's features.

32. Your agent remembers outdated information about a user. How would you manage memory effectively?

Agent memory is not a single thing. Effective memory architectures separate four layers and have policies for each.

Memory types

- **Working memory (session):** the current conversation. Cleared at the end of the session. Sized to fit the model's context window.
- **Episodic memory (events):** specific past interactions, retrievable by similarity. "Last Tuesday the user asked about X."
- **Semantic memory (facts):** structured facts about the user. "Lives in Bengaluru. Prefers vegetarian food. Works in EdTech."
- **Procedural memory (preferences):** how the user likes the agent to behave. "Always send summaries by email. Always use Indian English."

Update strategy

- **Extract on write:** after each conversation, a small LLM call extracts new facts, contradictions with existing memory, and preference updates. Persist them with timestamps and source-conversation IDs.
- **Detect contradictions:** if a new statement contradicts an existing fact, ask the user to confirm before overwriting, or mark the old fact as superseded with effective dates.
- **Decay and TTL:** facts that are inherently time-sensitive ("I am travelling next week") get short TTLs. Stable facts ("my name is X") do not.
- **User-controlled:** expose a "what does the assistant remember about me" view so users can review, edit, and delete. This is also a compliance requirement under DPDP and GDPR.

Retrieval strategy

- On each turn, retrieve only the memory relevant to the current message — embedding similarity on episodic memory, structured query on semantic memory.
- Never dump the entire memory into context. It blows the budget and dilutes signal.

Tools

- Mem0, Letta (formerly MemGPT), Zep, or a custom implementation over Postgres + a vector store.

Outdated information is almost always a write-path bug, not a read-path bug — the system noticed a change and did not record it. Fix the extractor before tuning retrieval.

33. An AI research assistant must browse the web and summarize findings. How would you design the agent architecture?

Web research agents are a popular but tricky use case. The architecture has to handle planning, source quality, anti-bot defences, and grounding.

Tools

- **Search:** a search API (Brave Search, Tavily, SerpAPI, Bing) that returns titles, snippets, URLs.
- **Fetch:** an HTTP client that downloads and parses pages, returning clean markdown. Handle JS-heavy pages via a headless browser (Playwright, Browserbase).
- **Read:** a chunker and reader that lets the agent ask questions of a long page without putting the whole thing in context.
- **Compare / cross-check:** a helper that asks "do these N sources agree on claim X?".

Workflow

1. **Decompose the question** into sub-questions with the planner.
2. **For each sub-question:** search → pick 3–5 candidate sources → fetch → extract relevant passages → summarise with citations.
3. **Cross-check** key facts across sources. Flag disagreements explicitly rather than averaging.
4. **Synthesise** into a final answer with inline citations and a sources list.

Source quality

- Maintain a tier list of sources by reliability for each domain (academic > established media > general web). Prefer higher tiers.
- Detect AI-generated content and de-weight it — much of the web is now LLM-generated and contains hallucinations.
- Filter by recency for time-sensitive queries.

Anti-hallucination

- Every claim in the final answer must be attributable to a fetched passage. If the model writes something not in the sources, drop it.
- Use a verifier model to check (claim, source) pairs before final output.

Operational

- Cache pages aggressively — research agents otherwise re-fetch the same URL many times.
- Respect robots.txt and rate limits.
- Budget: cap search calls, fetch calls, and total tokens per query.

- Stream partial results to the user — research takes time and users want progress.

34. An agent has access to internal company databases. What security measures would you implement?

This is the highest-risk class of agent. An LLM with raw SQL access on a customer database is a prompt injection away from disaster. Defence in depth is mandatory.

Identity and authorisation

- The agent acts as the requesting user, with their permissions — never as a privileged service account.
- Row-level security in the database, enforced by the database itself, not by application code.
- Column-level masking for sensitive fields (salary, PII).

Query interface

- Do not give the agent raw SQL. Give it parameterised tools: `get_customer(id)`, `list_orders(customer_id, date_range)`, `search_products(filters)`. Each tool encapsulates safe queries with bound parameters.
- If text-to-SQL is required, generate the SQL with the LLM, then parse and validate it (allow-list tables and columns, reject any DDL or DML, force LIMIT clauses) before execution.
- Read-only by default. Write tools are separate, named, and gated by additional confirmation.

Data handling

- Redact PII before passing rows to the LLM unless strictly necessary.
- Tokenise sensitive identifiers — the LLM sees "customer-7af3" not the real customer ID or email.
- Cap result set size returned to the LLM (e.g. 100 rows max) — prevents accidental data exfiltration via verbose output.

Prompt injection defence

- Treat any data retrieved from the database as untrusted text (a customer's note could contain "ignore previous instructions").
- Use instruction hierarchy — system prompt rules always win over content.
- Run a classifier on tool outputs to flag injection attempts.

Audit and monitoring

- Log every tool call: user, timestamp, parameters, rows returned, response.
- Anomaly detection: alert on unusual access patterns (e.g. a single user querying 10,000 customers in an hour).
- Retention policy: logs retained for the audit period required by regulation.

Operational guardrails

- Rate limit per user.
- Sandbox execution — even validated SQL runs in a read replica with a hard query timeout.
- Kill switch: an operations dashboard can disable the agent's database access immediately.

The single most important principle. An LLM has no concept of authorisation. Authorisation is enforced by the system around the LLM. Design accordingly.

Section 6 — Fine-Tuning and Customization

Fine-tuning is the most over-prescribed and under-evaluated technique in applied GenAI. Strong answers in this section show that you treat it as one option among several, with a clear evaluation discipline.

35. Your management team wants to fine-tune a model for customer support. How would you determine whether fine-tuning is actually necessary?

Push back politely. Fine-tuning has costs that the management team may not see: data preparation, training, evaluation, ongoing maintenance, vendor lock-in. Justify it only if cheaper options have been tried.

Decision sequence.

1. Baseline with prompting only. Take 100–300 representative tickets. Engineer the best system prompt with few-shot examples. Measure accuracy, tone, policy adherence. This is your baseline; you cannot improve what you have not measured.

2. Add RAG if knowledge gaps are the issue. Most support failures are not stylistic — they are factual. RAG over the help center, policy docs, and ticket history fixes the bulk of knowledge errors without any training. Measure again.

3. Consider fine-tuning only if the residual failure modes are stylistic or behavioural, not knowledge-based. Examples that justify FT: a specific tone the prompt cannot reliably reproduce, a structured output format the model keeps drifting from, very high call volume where switching to a smaller fine-tuned model would save substantial cost, or compliance requirements for a specific phrasing.

4. Quantify the gain. Estimate the lift from FT (typical: 5–15% on the relevant metric). Compare to the cost (data labelling, compute, MLOps). If the ROI period is over 12 months, FT is rarely worth it.

5. Choose the right FT technique. If you do proceed, LoRA / QLoRA is almost always the right starting point — cheaper, faster, and the base model's general capability is preserved.

How to communicate this. Present the team with a one-page matrix: option (prompt / RAG / FT / hybrid), expected lift, cost, time-to-deploy, ongoing maintenance. Let them choose with eyes open. "We should fine-tune" without this analysis is usually a mistake.

36. You have only 5,000 high-quality training examples. Would you fine-tune, use RAG, or rely on prompting? Why?

5,000 is a useful pivot point. With LoRA / QLoRA on a 7B–13B model, you have enough data to make a measurable difference — but you would still try cheaper options first.

Step 1 — Inspect the data. What do the 5,000 examples teach? If they are (question → factual answer with citation), they are RAG data, not FT data. If they are (input → specific format output) or (input → specific tone output), they are FT data.

Step 2 — Try few-shot. Pick the 5–10 best, most diverse examples. Use them as few-shots in the prompt. Measure on a held-out 500 examples. If accuracy is acceptable, you are done — no training, no infrastructure, no maintenance.

Step 3 — Try RAG over the 5K. Treat the examples as a knowledge base. Embed and retrieve the most similar past examples at inference time. This often outperforms FT for question-answering use cases and adapts to new examples without retraining.

Step 4 — If 1 and 2 are insufficient, fine-tune. 5K examples are enough for LoRA on a small model. Use parameter-efficient FT, validate carefully on a held-out set, and check for hallucination regression (see Question 37). Expect a 5–15% lift on the relevant metric.

Practical rule of thumb. Fine-tune for behaviour (style, format, refusal patterns). Use RAG for knowledge. Use prompting for both, first.

37. After fine-tuning, model accuracy improves but hallucinations increase. How would you address this?

This is a well-known regression mode — sometimes called "catastrophic forgetting" of the base model's calibration. Fix it from data, training, and evaluation.

Diagnose first. Are the hallucinations on topics outside the training distribution (model became overconfident in its narrow domain and now answers everything that way) or on the training topics themselves (training data contained errors or contradictions)?

Data fixes.

- Audit the training data for incorrect answers. Even 1–2% noisy labels can cause confident wrong answers.
- Add explicit "I don't know" examples. If the model never sees a refusal during training, it learns to always answer. Include 5–15% of examples where the correct response is to refuse or say insufficient information.
- Mix in generic instruction-following data. If you train only on your domain, the model loses its broader calibration. A 70/30 mix of domain and general data prevents this.

Training fixes.

- Use LoRA / QLoRA, not full fine-tuning. Adapters preserve the base model's behaviour outside the targeted distribution.
- Lower the learning rate.
- Train for fewer epochs. Most LoRA jobs over-train at 5+ epochs; stop at 1–3 with early stopping on a held-out set.
- Add a KL-divergence regulariser to the base model (DPO, ORPO) if the framework supports it.

Evaluation fixes.

- Evaluate hallucination as a separate metric alongside accuracy, not bundled into a single score. A model that is 2% more accurate but 8% more hallucinatory has regressed.
- Run out-of-distribution evals — questions intentionally outside the FT domain — to check the base capability has been preserved.
- Use a verifier model to score answer-context alignment.

Runtime fixes.

- Combine the fine-tuned model with RAG. Even after FT, ground factual claims in retrieved sources.
- Add output validation — citations, schema, self-check — before the answer reaches the user.

38. The fine-tuned model performs well on training data but poorly on unseen data. What could be wrong?

This is overfitting. Walk through the most common causes in order of likelihood.

- 1. Training set too narrow.** The examples cover a thin slice of the real distribution. The model memorises patterns that do not generalise. Fix: collect or augment data to cover the full distribution — different phrasings, edge cases, languages, lengths.
- 2. Too many epochs.** Especially common with small datasets. The model has memorised individual examples. Fix: early stopping on a held-out validation set; use 1–3 epochs as a default for LoRA.
- 3. Train / test leakage.** Test examples (or near-duplicates) appeared in training. Even paraphrases of training questions can leak. Fix: deduplicate aggressively, including semantic deduplication via embeddings, and split by source document or user where possible.
- 4. Hyperparameter mismatch.** Learning rate too high, batch size too small, rank too high for LoRA. Fix: hyperparameter sweep with a held-out validation set, not the test set.
- 5. Loss function does not match the metric.** Token-level cross-entropy can be optimised while end-to-end quality drops. Fix: evaluate with the metric you care about (task accuracy, judgement by a stronger model, human rating), not just loss.
- 6. Distribution shift between train and prod.** Training data was clean (one user, one annotator); production has typos, multiple languages, longer inputs. Fix: source training data from production-shaped examples; include adversarial and noisy variants.
- 7. Data quality issues.** Label noise, inconsistent annotation guidelines, biased sampling. Fix: an audit pass on a sample of training data with multiple annotators; measure inter-annotator agreement.

Process fixes for the long run. Maintain a held-out test set the model never sees. Re-evaluate every model version on it. Track the train/test gap as a first-class metric — when it widens, stop and investigate.

39. Your company wants a chatbot that understands internal terminology and acronyms. How would you customize the model?

Internal vocabulary is the cheapest customisation problem in GenAI. You almost never need fine-tuning for this.

Approach 1 — Glossary in the system prompt. Maintain a structured glossary (term → definition, with examples). Inject the relevant entries into the system prompt based on the query. For 50–200 terms, this fits comfortably and is essentially free to update.

Approach 2 — RAG over an internal wiki. If terms are documented in a Confluence or wiki page, RAG retrieves the definitions on demand. New terms appear in the chatbot the moment they appear in the wiki.

Approach 3 — Query expansion. Use a small LLM call (or a deterministic lookup) to expand acronyms in the user query before retrieval and generation. "What is the status of ARR for FY26?" becomes "What is the status of Annual Recurring Revenue (ARR) for Financial Year 2026?".

Approach 4 — Fine-tune only if needed. Reach for FT only when (a) the glossary is very large (thousands of terms with complex usage rules), (b) the model is consistently failing despite glossary injection, or (c) inference cost makes the glossary prohibitively expensive to include. In

that case, fine-tune on (user message with internal term, correct response that demonstrates understanding) pairs.

Tokenizer note. Some highly specific terms tokenise poorly (a 7-letter acronym becomes 4 tokens). For very high-volume cases, adding tokenizer entries (BPE merges) can help, but this is rarely worth the effort.

Practical pattern. Combine glossary injection with RAG. The glossary is fast and authoritative; RAG handles the long tail. Fine-tuning is a last resort.

40. You are asked to reduce fine-tuning costs by 80%. What techniques would you suggest?

An 80% reduction is realistic. Combine three or four of these techniques and you will get there.

1. Move to parameter-efficient fine-tuning. LoRA reduces trainable parameters by 100–1000x with minimal quality loss. QLoRA adds 4-bit quantisation of the base model, fitting a 70B model on a single 48 GB GPU. This is typically a 5–10x cost reduction on its own.

2. Use a smaller base model. Most use cases that work on a 70B model also work on a well-prompted 8B model after FT. Llama 3.1 8B, Qwen 2.5 7B, Phi-4 are strong starting points. Cost drops by an order of magnitude across training and inference.

3. Reduce the dataset. Quality dominates quantity. Curate 1–2K best examples instead of training on 20K. Use clustering / diversity sampling to keep coverage. Training time drops proportionally.

4. Use spot / preemptible GPUs. On AWS, GCP, or Lambda Labs, spot GPUs are 50–70% cheaper. Combined with checkpointing every N steps, the interruption risk is manageable.

5. Mixed precision and gradient checkpointing. bf16 / fp16 mixed precision cuts memory and time. Gradient checkpointing trades compute for memory, allowing larger effective batch sizes.

6. Use managed FT services. Together AI, Fireworks, Modal, Anyscale, and the major cloud providers offer FT services that handle infrastructure efficiently. For one-off jobs they are often cheaper than self-managed.

7. Avoid re-training from scratch. Continue training an existing adapter when new data arrives rather than redoing the full job. Cache and version adapters.

8. Distillation. Use a large model to label or generate data, then fine-tune a small model on that. The small model captures most of the large model's behaviour at a fraction of the inference cost. This is often what "80% cheaper" actually looks like in production.

Section 7 — Deployment and Production Systems

This section moves from "the model works on my laptop" to "the model works for ten thousand users in three regions at 2 AM on a Sunday." The interviewer is looking for operational maturity — scaling, observability, cost control, and incident response.

41. Your GenAI API handles 100 requests per minute today but is expected to handle 10,000 requests per minute next month. How would you scale it?

100x in a month is aggressive. Plan the scale-up in layers and ship in stages.

Layer 1 — Capacity

- **If using a managed provider (OpenAI, Anthropic, Bedrock, Vertex):** request rate limit increases now, not on launch day. Negotiate provisioned throughput for predictable load.
- **If self-hosting:** scale the inference fleet horizontally. Use vLLM or TGI for GPU-efficient serving with continuous batching. A single A100 80 GB can serve hundreds of QPS on a 7–8B model with vLLM.
- **Autoscaling:** Kubernetes HPA based on queue depth or GPU utilisation, not CPU.
- **Multi-region:** deploy in 2–3 regions for both capacity and latency. Use latency-based routing.

Layer 2 — Throughput optimisation

- **Continuous batching** in the inference server: requests are dynamically batched at the token level, not at the request level. This is the single biggest throughput win for self-hosted models.
- **KV cache reuse:** for repeated system prompts (very common), use prompt caching where supported. Anthropic and Google offer this natively.
- **Speculative decoding** for latency-sensitive paths.
- **Streaming responses** reduces perceived latency and frees server resources earlier.

Layer 3 — Caching

- **Semantic cache:** embed the incoming query, look up similar past queries with high similarity, return the cached response. Cuts traffic to the LLM by 30–60% in typical support workloads.
- **Static response cache** for exact-match popular queries (FAQ, greetings).
- **CDN** for non-personalised assets.

Layer 4 — Architecture

- **Async queue (Kafka, SQS)** for non-realtime requests. The API accepts the request, queues it, and the client polls or receives a webhook. Smooths spikes.
- **Tiered models:** small model handles 70% of requests, large model handles the rest. Routes by classifier or first-pass attempt.
- **Rate limiting and queueing** at the API gateway — protect upstream from runaway clients.

Layer 5 — Cost discipline

- Monitor cost per request, per tenant, per route. Set budgets.
- Alarms before you hit provider rate limits.
- Reserved capacity for the baseline, on-demand for the spike.

Rollout plan

- Week 1–2: load test current system at 10x, identify bottlenecks.
- Week 2–3: deploy caching, batching, model routing.
- Week 3: scale infrastructure, request quota increases.
- Week 4: load test at 12,000 RPM, dark-launch real traffic via shadow mode.
- Launch: gradual ramp 10% → 50% → 100% with rollback ready.

42. A deployed chatbot suddenly experiences a 5-second latency increase. How would you investigate?

This is an incident. Restore service first if it is user-visible, then root-cause.

Step 1 — Localise. Check the latency breakdown in the trace: input processing, retrieval, model call, tool calls, response assembly. Which stage added the 5 seconds?

Step 2 — Recent changes. Look at the last 24 hours of deploys, config changes, and feature flag flips. "Sudden" almost always correlates with a change.

Step 3 — Provider status. Check the LLM provider's status page. Hosted model latency can spike with provider incidents. Often the answer is here.

Step 4 — Inspect the model call.

- Token count: are prompts longer now (RAG returning more chunks, longer conversation history)?
- Model version: a silent model upgrade can change latency.
- Region: are calls being routed to a more distant region?
- Streaming: is streaming still enabled, or did a code change disable it?

Step 5 — Inspect the retrieval. Vector DB latency (see Question 16). A slow retrieval adds time to every request.

Step 6 — Inspect tools. External API calls that the agent makes — Salesforce, Slack, etc. — may be slow or rate-limited.

Step 7 — Infrastructure. Container CPU/memory, GC pauses, connection pool exhaustion, DNS resolution issues, network saturation between services.

Step 8 — Load. Traffic spike causing queueing? Check QPS and queue depth metrics.

Mitigation while investigating. Roll back the most recent deploy, shed non-critical load, redirect traffic to a healthy region, or reduce the model from large to small as a temporary fallback.

Post-incident. Write the incident up: timeline, root cause, what would have caught it earlier. Add the missing alert and the missing dashboard. Most repeated incidents share a missing observable.

43. Your GenAI service works locally but fails after Docker deployment. How would you troubleshoot?

This is one of the most common ship-day problems and almost always traces to environment differences, not code.

Step 1 — Read the container logs. Surprisingly often, the answer is in the first 50 lines: a stack trace, a missing env var, a connection refused. Start there before guessing.

Step 2 — Walk through the differences:

- **Environment variables:** API keys, model names, endpoint URLs. The .env loaded locally is not in the container by default.
- **File paths:** absolute paths that exist locally (`/Users/sudhanshu/data/...`) do not exist in the container. Use relative paths or mounted volumes.
- **Dependencies:** the local environment has packages the container does not. Audit `requirements.txt` or `package.json` — pin versions; do not use range specifiers.
- **Python / Node versions:** the container's interpreter may differ from local.
- **System libraries:** some Python packages need `libssl`, `libgomp`, etc. Add to the Dockerfile.
- **Architecture:** M-series Mac builds are `arm64`; cloud usually runs `amd64`. Build with `--platform linux/amd64` or use multi-arch images.

Step 3 — Network.

- DNS in the container — can it resolve `api.anthropic.com`?
- Egress rules — the corporate proxy or VPC may block the outbound call.
- Port mapping — the API listens on 8080 inside, but is the host port mapped?

Step 4 — Permissions.

- Files mounted as volumes may have wrong permissions.
- Running as a non-root user that cannot bind to privileged ports.

Step 5 — Resources.

- Container memory limit too low → OOM kill.
- CPU limits → timeouts on otherwise-fast operations.

Step 6 — Reproduce locally with Docker. Build and run the same image locally. If it works locally in Docker but fails in cloud, the problem is environment (cloud env vars, IAM, network). If it fails locally in Docker too, the problem is in the image.

Prevention. Use a dev container that matches production. Run integration tests in CI inside the image. Maintain a parity checklist for env vars between local and cloud.

44. A production LLM application is consuming excessive API credits. How would you optimize costs?

Start by understanding where the spend goes. Without measurement, optimisations are guesswork.

Step 1 — Profile. Break down spend by input tokens, output tokens, model, route, tenant, and time of day. Most teams find 80% of spend in 20% of traffic — a few endpoints, a few power users.

Step 2 — Attack input tokens first. In most apps, input tokens are 3–10x more expensive than output.

- Trim the system prompt — every kilobyte ships with every call.

- RAG with fewer, better chunks — a reranker that returns 3 chunks of 500 tokens each is far cheaper than 20 chunks of 800.
- Conversation history truncation — summarise older turns instead of including verbatim.
- Prompt caching — Anthropic, Google, and OpenAI now support caching of common prefixes; can cut input cost by 50–90% on cache hits.

Step 3 — Attack output tokens.

- Cap max_tokens — many models will write essays if allowed; set realistic limits.
- Structured outputs — JSON forces brevity, prose meanders.
- Stop sequences.

Step 4 — Route smarter.

- Cascading: classify intent with a cheap model, escalate only when needed.
- Use a smaller model where it suffices — most chat traffic does not need the flagship.
- Per-tenant model selection — premium tenants get the big model, free tier gets the cheap one.

Step 5 — Cache.

- Exact-match cache for FAQs.
- Semantic cache for paraphrased queries.
- Conversation-level cache for repeated tool outputs.

Step 6 — Batch APIs. Anthropic and OpenAI batch APIs offer ~50% discount with 24-hour SLA. Use for any non-realtime work — summaries, evals, offline ingestion.

Step 7 — Budget controls.

- Per-tenant rate limits and spend caps.
- Alerts at 50% / 75% / 90% of monthly budget.
- Automatic throttling when nearing cap.

A well-instrumented cost programme typically cuts spend by 50–70% within a quarter without any quality loss — most of the spend is structural waste, not necessary cost.

45. Users across different countries experience different response times. How would you solve this problem?

Latency variance by geography is almost always a routing / proximity problem. Solve it at the network edge.

Diagnose. Measure end-to-end latency per region with synthetic monitors. Break down: client → API gateway, gateway → application, application → LLM provider, application → vector DB, application → tools. Find which segment varies by country.

Common causes.

- Single-region LLM endpoint: a user in India calling a us-east-1 endpoint pays 200–300 ms RTT before the model even starts.
- Single-region application: similar issue at the application layer.
- Vector DB in one region only.
- DNS or CDN edge serving from a distant POP.

Solutions.

- **Multi-region deployment** of the application (us-east, eu-west, ap-south as a baseline). Latency-based routing (Route 53, Cloud DNS) sends users to the nearest healthy region.
- **Regional LLM endpoints.** Azure OpenAI offers region-specific deployments; Bedrock has regional endpoints; Vertex too. Pin the application to call the LLM in its own region.
- **Replicate the vector DB** or use a managed service with regional read replicas (Pinecone regional pods, Weaviate clusters).
- **CDN for static assets** (JS, CSS, images) — fronts a global edge.
- **WebSocket / SSE endpoints** should also be regionalised; otherwise streaming traffic suffers.
- **Edge compute** (Cloudflare Workers, AWS Lambda@Edge) for lightweight pre-processing close to the user.

Operational. Per-region dashboards. Synthetic monitors in each major user country. SLAs defined per region, not globally.

46. During deployment, a new model version causes answer quality to drop. How would you manage rollback and versioning?

This is preventable. The deploy strategy should make it almost impossible to ship a quality regression without noticing.

Versioning

- Every deployable unit — the prompt template, the model identifier, the retrieval config, the tool schemas — has an explicit version.
- Versions are stored in a registry (MLflow, a config service, or a Git-versioned config repo).
- Every production response logs the full bundle version used.

Pre-deploy gates

- **Offline eval.** Run the new bundle against a curated eval set (a few hundred to a few thousand examples covering the distribution and known hard cases). Score on accuracy, safety, format adherence.
- **Quality gate.** Deploy only if the new bundle scores within X% of the previous on every key metric. No bulk averages — individual metrics matter.
- **Shadow mode.** Mirror production traffic to the new bundle for 24–72 hours without serving the responses. Compare against the production response. Inspect a sample manually.

Rollout

- **Canary.** Route 1% of traffic to the new bundle. Monitor quality signals (user feedback, deflection rate, follow-up rate). If clean for an hour, go to 5%, 25%, 100% over a day.
- **Auto-rollback.** Set thresholds: if response latency, error rate, or a quality proxy (e.g. user thumbs-down rate) crosses a limit, the deploy automatically rolls back.
- **Per-tenant rollouts** for B2B platforms — start with internal tenants, then friendly customers, then GA.

Incident handling when a regression escapes

- Rollback first — usually one switch in the config or registry.
- Capture diff: what changed (model, prompt, retrieval, tools)?

- Build a regression test from the failures so the same bug cannot ship again.
- Communicate: status page update, incident review.

Cultural point. Treat prompts and configs as code — versioned, reviewed, tested. Treat eval sets as living assets — they grow whenever you encounter a new failure mode.

Section 8 — System Design, Monitoring, and Guardrails

This section is where senior interviews are won or lost. It is no longer about LLMs in isolation but about engineering responsible, observable, secure systems around them. The strongest answers stop trying to make the model perfect and instead build a system where imperfect models are still safe and useful.

47. Design an AI-powered interview platform that can handle 100,000 concurrent candidates. What architecture would you propose?

100,000 concurrent interviews is a non-trivial real-time system. The architecture has four core domains: candidate experience, AI inference, data, and operations. Each must scale independently.

Candidate experience layer

- Web client: React or Next.js, with WebRTC for live video / audio capture.
- Mobile client: native iOS / Android, or React Native, for candidates without reliable desktop access.
- Real-time channel: WebSocket or Server-Sent Events for streaming interviewer prompts and feedback.
- CDN: static assets, candidate landing pages, completed report exports.

Voice and language pipeline

- Speech-to-text: a low-latency streaming STT (Deepgram, Whisper Large v3 served via streaming, or a cloud provider) — running per-session in the candidate's nearest region.
- Text-to-speech: streaming TTS so the interviewer voice begins playing within 300–500 ms (ElevenLabs, Azure Neural TTS, or a self-hosted alternative).
- Language detection and routing — Indian candidates may answer in code-mixed Hinglish; the pipeline handles this.

Session and state

- Per-session state lives in Redis: current question, history, candidate answers, model judgements so far.
- Persistence layer (Postgres) for the interview record once committed.
- Recordings to S3 with KMS encryption, lifecycle policies (move to Glacier after 90 days).

Anti-cheat / proctoring

- Webcam analysis (face presence, gaze, multiple persons).
- Screen / tab change detection on the client, reported as events.
- Audio fingerprinting to flag impersonation.
- Behavioural signals: typing cadence, paste detection, answer latency.

Scoring and reporting

- Scoring runs in two phases: live (lightweight signals during the interview) and post-interview (a stronger model produces the structured report).

- Reports follow a fixed schema (see Question 11) and are reviewable by humans.

Operations

- Per-region deployments with autoscaling on GPU pool, app fleet, and Redis.
- Queue (Kafka) for events; consumers handle reporting, billing, compliance.
- Observability: per-tenant dashboards, p95 latency by stage, GPU utilisation, error budgets.
- SRE on-call with documented runbooks.

Compliance

- Candidate consent recorded before recording starts (GDPR, DPDP).
- Data residency by client jurisdiction (EU candidates served from EU region).
- Bias monitoring on scoring — disparate impact tests by gender, age band, region.
- Right to deletion fulfilled by tenant-scoped delete jobs.

Capacity sketch

- 100,000 concurrent sessions × 1 turn every 10–20 seconds = 5,000–10,000 inference QPS at peak.
- At 30 input + 100 output tokens per turn, throughput is well within range for a vLLM fleet of ~100–200 A100s split across regions, plus a smaller pool for STT and TTS.
- Plan for 3x peak to absorb spikes (job application deadlines, walk-in drives).

48. Your chatbot starts leaking sensitive customer information in responses. What guardrails would you implement immediately?

Treat this as a P0 incident. Stop the leak first; fix the root cause second; harden the system third.

Immediate — stop the leak (within 1 hour)

- Pause the affected route or downgrade it to a safe canned response.
- Deploy a strict output filter that detects PII patterns (emails, phone numbers, account numbers, government IDs) and either redacts or refuses.
- Tighten the system prompt: "You must never include any personally identifiable information in your response, even if it appears in the provided context. If a user asks for PII, refuse."
- Communicate to affected users where required by law.

Same day — diagnose the source

- Where is the PII coming from? RAG context pulling another user's data? A tool returning a wider result set than intended? Conversation memory contaminating across sessions? The model fabricating realistic-looking PII?
- Inspect 50–100 leak samples to classify.

Week one — root cause fix

- **RAG ACL:** verify and enforce per-user permissions on the vector store. Filter at query time using the user's identity, not at index time only.
- **Tool boundary:** audit each tool's return shape. Strip fields the chatbot does not need.
- **Session isolation:** every session has a unique key. Memory and caches are partitioned by tenant + user.

- **Input filter:** PII detection on inbound messages too — if the user is feeding PII into the system, you may be storing it in places that leak it later.
- **Output filter:** production-grade PII redaction (Presidio, Lakera Guard, NeMo Guardrails, or a custom Llama Guard fine-tune) running on every response.
- **Prompt redesign:** ensure the system prompt cannot be coaxed into revealing context via prompt injection (see Question 49).

Ongoing — prevent recurrence

- Red-team eval set covering known PII-leak attack patterns; runs on every prompt or model change.
- Audit logs of redactions — if the filter is firing, something upstream is producing PII it should not.
- DLP scan on stored conversations.
- Annual penetration test focused on the GenAI surface.

Communications. Internal incident review must be honest about the root cause. External communications follow legal counsel and the applicable breach-notification law.

49. A malicious user attempts prompt injection to access confidential company data. How would you defend against it?

There is no single defence. Prompt injection is mitigated, not eliminated, and the defence is always layered.

Layer 1 — Input boundary

- Treat all user input and all retrieved content as untrusted data, never as instructions.
- Use clear delimiters around user input (XML tags or a distinct format) and instruct the model that nothing inside those delimiters is an instruction.
- Run an input classifier (Lakera, Rebuff, Llama Guard, a custom-trained model) that scores the probability of injection. Block or sandbox high-scoring inputs.

Layer 2 — System prompt design

- Establish an instruction hierarchy — system instructions always win over user content. State this explicitly and repeatedly.
- Spotlighting — mark user content with a special token or wrapper that the model is trained to recognise as data.
- Never include sensitive secrets in the system prompt. The system prompt is reachable through prompt-injection extraction. Secrets belong in tool implementations, not in text.

Layer 3 — Tool and data boundary

- Least privilege on tools — the model can only access what the requesting user can.
- No tool returns confidential bulk data. Tools return scoped, minimal results.
- Destructive tools require user confirmation, not just model intent.
- PII redaction on tool outputs before they reach the model.

Layer 4 — Output boundary

- Output filter scans for sensitive patterns (secrets, internal URLs, PII).

- Schema validation — if the response shape is unexpected, reject.
- Secondary judge model: "Does this response contain confidential information that should not be shared with the user?"

Layer 5 — Detection and response

- Anomaly detection on user behaviour — unusual phrasing patterns, repeated probing prompts.
- Honeytokens — fake secrets in the system that, if leaked, trigger an alert. If you ever see them in a response, you have been pwned.
- Audit logs of injection attempts feed back into the input classifier.

Layer 6 — Organisational

- Red-team the system regularly with both automated tools and human attackers.
- Disclosure programme for researchers to report findings.
- Incident response runbook for confirmed injection-based breaches.

Senior interview point. Acknowledge the open problem honestly. Even frontier labs have not solved prompt injection. The mitigation is to assume the model can be subverted and ensure that subversion cannot translate into business harm — by constraining what the model is allowed to do, not by hoping it always refuses.

50. Your CEO asks you to design a production-grade enterprise GenAI platform with RAG, agents, monitoring, security, cost controls, and multi-model support. Walk through the complete architecture you would propose.

This is the capstone question. A strong answer presents the platform as a set of clearly-bounded planes with explicit responsibilities, then describes how they interact. The aim is not to dump features but to demonstrate that you can architect for change — new models, new tenants, new use cases — without rewriting.

Guiding principles

- Model-agnostic: any closed or open model can be plugged in behind a single gateway.
- Multi-tenant: strict logical isolation between tenants; per-tenant configuration, quotas, and observability.
- Defence in depth: no single component is the only thing protecting the user, the data, or the business.
- Observable: every decision the system makes is logged, attributable, and reviewable.
- Cost-aware: every request has a known cost, and every tenant has a known budget.

Control plane

- **Identity and access:** SSO (SAML / OIDC), SCIM provisioning, RBAC and ABAC. Tenant admins manage users and roles.
- **Tenant management:** tenant catalogue, configuration store, feature flags, model entitlements (which models a tenant can use).
- **Policy engine:** expresses data residency, allowed models, content policies, audit retention. Enforced consistently across the data plane.

- **Cost engine:** budgets, quotas, rate limits per tenant and per team. Alerts and hard caps.
- **Audit and compliance:** immutable logs of authentication, configuration changes, and sensitive operations.

Data plane

- **Model gateway:** a single internal API (LiteLLM-style) that fronts all model providers — OpenAI, Anthropic, Bedrock, Vertex, self-hosted vLLM clusters. Handles routing, fallback, retries, key management, and per-tenant model selection. This is the single most important abstraction in the platform.
- **Prompt and template registry:** versioned prompts, with environment promotion (dev → staging → prod), associated eval sets, and rollback. Treated as code.
- **RAG layer:** a managed retrieval service. Connectors for SharePoint, Confluence, Google Drive, Slack, internal databases. Common ingestion, chunking, embedding, and indexing. Per-tenant vector partitions. ACL enforced at query time. Hybrid search + reranker.
- **Agent runtime:** LangGraph or a custom orchestrator. Hosts agents as graph definitions with checkpointing, retries, and observability. Tool registry — every tool has a schema, an owner, an SLA, and a permission scope.
- **Memory store:** session, episodic, semantic, and procedural memory per user, with TTLs, contradiction detection, and user-controlled deletion.
- **Guardrails service:** input classifiers, output filters, PII redactors, refusal policies. Applied uniformly across all routes.
- **Cache:** exact and semantic caches at the gateway level and at the application level.

Observability and quality plane

- **Tracing:** every request emits a trace covering retrieval, model calls, tool calls, guardrails. OpenTelemetry-based; viewable in Langfuse, LangSmith, Helicone, or similar.
- **Metrics:** p50/p95 latency, error rate, token usage, cost, retrieval recall on a sampled eval, guardrail trigger rate. Dashboards per tenant and per route.
- **Evaluation pipeline:** an automated eval runner triggered on every prompt or model change. Eval sets are versioned alongside prompts. Quality gates block bad deploys.
- **Human review queues:** low-confidence responses, flagged conversations, and a sampled stream go to human reviewers. Their judgements train better classifiers and improve eval sets.
- **Cost reporting:** per-tenant, per-route, per-model spend, with chargeback to internal cost centres.

Deployment and infrastructure

- Kubernetes for application services, with autoscaling on application metrics.
- GPU pools (managed and self-hosted) for self-hosted models, with vLLM / TGI.
- Multi-region deployment for latency and resilience.
- Streaming everywhere — both for UX and for resource efficiency.
- CI/CD pipelines that include eval gates, security scans, and shadow-mode deployment for prompt and model changes.

Security

- Network: VPC isolation, private endpoints to model providers, no LLM traffic over the public internet for sensitive tenants.
- Encryption: at rest (KMS), in transit (TLS 1.3), and tenant-scoped keys for sensitive workloads.
- Secrets: a secrets manager, not env vars; short-lived credentials.
- Prompt-injection defence (see Question 49) applied uniformly.
- Vulnerability management: model providers' security disclosures tracked; SBOM for self-hosted models.

Multi-model strategy

- Frontier closed-source models for reasoning-heavy tasks (Claude, GPT, Gemini).
- Strong open models on self-hosted infra (Llama 3.x, Qwen) for bulk traffic and cost-sensitive routes.
- Domain-tuned smaller models (LoRA on a 7B base) for specific tasks at high volume.
- Routing decisions made at the gateway, based on task class, tenant tier, and live cost/latency telemetry.

Build / buy / open-source

- Buy: identity (Okta / Azure AD), observability backends (Datadog or open-source Grafana stack), some guardrail components (Lakera).
- Open-source assemble: gateway (LiteLLM), agents (LangGraph), evals (Promptfoo, RAGAS, Langfuse), vector DB (Qdrant / pgvector).
- Build in-house: the policy engine, the cost engine, the tenant management surface, the prompt and eval lifecycle — these are where competitive advantage lives, and where off-the-shelf solutions are still immature.

Rollout sequencing

- Phase 1 (Quarter 1): Gateway + RAG + observability + one customer-facing use case.
- Phase 2 (Quarter 2): Agent runtime, tool registry, expand to 3–5 use cases.
- Phase 3 (Quarter 3): Cost engine, multi-region, self-hosted models for cost reduction.
- Phase 4 (Quarter 4): Self-serve tenant onboarding, marketplace of prompts and tools, fine-tuning pipeline.

Closing point. The architecture is deliberately boring. The interesting work — prompts, agents, evals, fine-tunes — happens on top of a stable, observable, governed platform. The platform is what lets a company ship the next ten use cases in weeks instead of quarters, and it is what makes the difference between a GenAI demo and a GenAI business.